



OASIS Security Report

SQL Injection Security Analysis

Date: 2026-06-11 10:45:56 — **Model:** mistral:latest

Vulnerability: SQL Injection

Project: clean_contracts

Analysis root: `../clean`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **2** file(s) for SQL Injection.

[Return to table of contents](#)

Statistics

Files analyzed: 2 **Total findings:** 2

Critical		0
High		1

Medium**1****Low****0**[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/clean/modifier_reentrancy_fixed.sol	0.574
/work/project/dataset/clean/simple_dao_fixed.sol	0.564

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.574

Source lines: 1-42

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
'keccak256(abi.encodePacked("Nu Token")) == bank.supportsToken()';
```

Explanation

The code uses a string directly in the SQL query, potentially allowing an attacker to inject malicious SQL.

Impact

Data theft, data loss, or authentication bypass

Entry point

supportsToken() function

Execution path

```
1 → supplies input → supportsToken() → SQL query execution
```

Parameters

bank.supportsToken()

Exploitation steps

- Inject malicious SQL in the manipulable parameter
- Execute the SQL query with the injected input

Example payloads

- 'Nu Token'; DROP TABLE users; --

Conditions

None

Remediation

Use parameterized queries or prepared statements

Secure example

```
'keccak256(abi.encodePacked("Nu Token")); querySupportsToken(...);'
```

File 2: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.564

Source lines: 1-34

► Finding 2.1: SQL Injection vulnerability (High)

Vulnerable code

```
'donate(to)' function accepts user-supplied 'to' parameter directly in SQL query
```

Explanation

User input is not properly sanitized before inclusion in the SQL query, allowing an attacker to inject malicious SQL statements and potentially steal data or manipulate system functionality.

Impact

Data theft, unauthorized access, or system compromise

Entry point

donate(to) function

Execution path

```
```markdown
> User -> donate(to)
|
^-SQL injection-
< Database```
```

#### Parameters

to

#### Exploitation steps

- Send a malicious payload to the 'to' parameter in the donate function
- Inject SQL commands into the query
- Execute arbitrary SQL commands on the database

#### Example payloads

- `' OR 1=1 --`

#### Conditions

None, but proper input validation and sanitization is crucial to mitigate this vulnerability.

## Remediation

Use parameterized queries or prepared statements to prevent SQL Injection attacks

### Secure example

```
```solidity
function donate(address to) public {
    require(!isZeroAddress(to), "Invalid address provided.");
    credit[to] += msg.value;
}
```
```

[Return to table of contents](#)

## Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)